# FocusOn — Documentación Técnica

> Aplicación móvil de gestión de tareas diarias construida con **Expo + React Native**, arquitectura de fichero plano y base de datos SQLite local.

---

## Índice


1. Stack tecnológico
2. Estructura de carpetas
3. Base de datos
4. Estado global — Zustand Store
5. Sistema de notificaciones
6. Pantallas (Tabs)
7. Modales de tarea
8. Componentes reutilizables
9. Sistema de diseño
10. Flujos principales
11. Cómo extender la app


---

## 1. Stack tecnológico

| Tecnología | Versión | Para qué se usa |
|---|---|---|
| **Expo** | ~54 | Framework base, build tools, plugins nativos |
| **React Native** | 0.81 | Renderizado de UI nativa |
| **Expo Router** | ~6 | Navegación basada en sistema de ficheros |
| **expo-sqlite** | ~16 | Base de datos local SQLite (nativo) |
| **AsyncStorage** | 2.2 | Fallback en web en lugar de SQLite |
| **Zustand** | ^5 | Estado global reactivo sin boilerplate |
| **expo-notifications** | ~0.32 | Notificaciones locales programadas |
| **date-fns** | ^4 | Formato y manipulación de fechas |
| **TypeScript** | ~5.9 | Tipado estático |

---

## 2. Estructura de carpetas

```
FocusOn/
├── app/                        ← Rutas de Expo Router
│   ├── _layout.tsx             ← Layout raíz: bootstrap, Stack de navegación
│   ├── (tabs)/                 ← Grupo de pestañas (tab bar)
```

```
│   │   ├── _layout.tsx         ← Configuración del tab bar (iconos, colores)
│   │   ├── index.tsx           ← Pestaña "Hoy"
│   │   ├── tasks.tsx           ← Pestaña "Tareas" (todas)
│   │   ├── stats.tsx           ← Pestaña "Stats" (estadísticas)
│   │   └── settings.tsx        ← Pestaña "Config"
│   └── task/
│       ├── new.tsx             ← Modal: crear tarea
│       └── [id].tsx            ← Modal: editar tarea (ruta dinámica)
│
├── components/                 ← Componentes reutilizables
│   ├── TaskCard.tsx            ← Tarjeta de tarea con swipe-to-delete
│   ├── AddTaskFAB.tsx          ← Botón flotante "+"
│   ├── ProgressBar.tsx         ← Barra de progreso animada
│   ├── EmptyState.tsx          ← Pantalla vacía con icono y mensaje
│   └── PriorityBadge.tsx       ← Pastilla de prioridad (Alta/Media/Baja)
│
├── store/
│   └── useTaskStore.ts         ← Store de Zustand (toda la lógica de negocio)
│
├── lib/
│   ├── db.ts                   ← Capa de datos SQLite (nativo)
│   ├── db.web.ts               ← Capa de datos AsyncStorage (web fallback)
│   └── notifications.ts        ← Permisos, programación y cancelación de notifs
│
├── constants/
│   ├── theme.ts                ← Tokens de diseño: Colors, Spacing, FontSize...
│   └── Colors.ts               ← Archivo legado (superseded por theme.ts)
│
└── assets/                     ← Imágenes, iconos, splash screen
```

> **Regla de oro de Expo Router:** cada fichero dentro de `app/` se convierte automáticamente en una ruta. Los grupos entre paréntesis como `(tabs)` no aparecen en la URL.

---

# 3. Base de datos

## 3.1 Esquema — tabla `tasks`

```
CREATE TABLE IF NOT EXISTS tasks (
  id              INTEGER PRIMARY KEY AUTOINCREMENT,
  title           TEXT    NOT NULL,
  description     TEXT    DEFAULT '',
  priority        TEXT    DEFAULT 'medium',   -- 'high' | 'medium' | 'low'
  category        TEXT    DEFAULT 'general',  -- 'general' | 'trabajo' | 'salud' |
'personal'
  scheduled_date  TEXT    DEFAULT (date('now')), -- 'YYYY-MM-DD': día asignado a
la tarea
  reminder_time   TEXT,                       -- 'HH:mm' o NULL
  repeat_minutes  INTEGER DEFAULT 0,          -- 0 = sin repetición
  is_completed    INTEGER DEFAULT 0,          -- 0 | 1 (booleano SQLite)
  completed_at    TEXT,                       -- ISO timestamp o NULL
```

```
  created_at      TEXT    DEFAULT CURRENT_TIMESTAMP,
  notification_id TEXT                        -- ID de la notif programada, o NULL
);
```

### 3.2 Tipos TypeScript (`lib/db.ts`)

```ts
export type Priority = 'high' | 'medium' | 'low';
export type Category = 'general' | 'trabajo' | 'salud' | 'personal';

export interface Task { /* todos los campos de la tabla */ }

// Para crear una tarea: omite id, timestamps y notification_id
export type NewTask = Omit<Task, 'id' | 'created_at' | 'completed_at' |
'is_completed' | 'notification_id'>;
```

### 3.3 Funciones disponibles (`lib/db.ts`)

| Función | Descripción |
|---|---|
| `initDB()` | Crea la tabla si no existe. Migra columnas nuevas con `ALTER TABLE`. Llamar una sola vez en el bootstrap. |
| `getAllTasks()` | Devuelve **todas** las tareas ordenadas por: pendientes primero → prioridad → hora recordatorio → fecha creación. |
| `getTasksByDate(date)` | Devuelve solo las tareas de un día concreto (`'YYYY-MM-DD'`). Usado por la pestaña "Hoy". |
| `getTaskById(id)` | Devuelve una tarea por ID o `null`. |
| `createTask(task)` | Inserta una nueva tarea. Devuelve el `id` generado. |
| `updateTask(id, fields)` | Actualiza los campos indicados de una tarea. |
| `deleteTask(id)` | Elimina una tarea por ID. |
| `toggleTaskComplete(id, completed)` | Marca/desmarca como completada y actualiza `completed_at`. |
| `resetAllTasks()` | Pone todas las tareas como pendientes (sin eliminarlas). |
| `deleteAllTasks()` | Elimina todas las tareas de la base de datos. |

> **Web fallback (`lib/db.web.ts`):** En lugar de SQLite usa un array en memoria persistido en AsyncStorage. La API es idéntica, por eso el store funciona igual en web y en nativo.

---

## 4. Estado global — Zustand Store

**Fichero:** `store/useTaskStore.ts`

El store es el **único punto de verdad** de la aplicación. Todos los componentes leen y escriben a través de él.

### 4.1 Estado

```
{
  tasks: Task[];          // array de tareas actualmente cargadas
  filter: FilterType;     // filtro activo en la pestaña "Tareas"
  categoryFilter: CategoryFilter;
}
```

### 4.2 Métodos

| Método | Descripción |
|---|---|
| `loadTasks()` | Carga **todas** las tareas en el store. Usado por Stats y Settings. |
| `loadTasksForDate(date)` | Carga solo las tareas del día `date`. Usado por la pestaña "Hoy". |
| `addTask(newTask)` | Crea una tarea en la DB y programa su notificación si tiene `reminder_time`. |
| `editTask(id, updates)` | Actualiza la tarea. Si cambia el título o el recordatorio, cancela la notif vieja y programa una nueva. |
| `removeTask(id)` | Cancela notificación y elimina la tarea. |
| `toggleTask(id)` | Completa/revierte una tarea. Al completar cancela la notif; al revertir la reprograma. |
| `resetDay()` | Desmarca todas las tareas y reprograma sus notificaciones. |
| `purgeAll()` | Cancela todas las notificaciones y borra todas las tareas. |
| `setFilter(filter)` | Cambia el filtro de estado activo. |
| `setCategoryFilter(cat)` | Cambia el filtro de categoría activo. |
| `getFilteredTasks()` | Devuelve las tareas del array `tasks` aplicando los filtros activos. |
| `getStats()` | Devuelve `{ total, completed, pending }`. |

### 4.3 Tipos de filtro

```
type FilterType     = 'all' | 'pending' | 'completed' | 'high' | 'medium' | 'low';
type CategoryFilter = 'all' | 'general' | 'trabajo' | 'salud' | 'personal';
```

---

## 5. Sistema de notificaciones

**Fichero:** `lib/notifications.ts`

### 5.1 Flujo de permisos

En `app/_layout.tsx`, durante el bootstrap, se llama a `requestNotificationPermissions()`. En Android además se crea el canal `focuson-tasks` con prioridad HIGH.

### 5.2 Programar una notificación

```
scheduleTaskNotification(task: Task): Promise<string | null>
```

- Requiere que `task.reminder_time` (`'HH:mm'`) y `task.scheduled_date` (`'YYYY-MM-DD'`) estén rellenos.
- Combina ambos en un `Date` exacto y usa el trigger de tipo **DATE** (dispara una única vez).
- Si la fecha/hora ya ha pasado, **no programa** la notif y devuelve `null`.
- El cuerpo del mensaje es aleatorio entre 9 frases motivacionales.
- Devuelve el `notification_id` (string) que se guarda en la DB para poder cancelarla.

### 5.3 Cancelar una notificación

```
cancelNotification(notificationId: string | null): Promise<void>
cancelAllNotifications(): Promise<void>
```

---

## 6. Pantallas (Tabs)

### 6.1 `app/(tabs)/index.tsx` — Pestaña "Hoy"

**Qué hace:** Muestra las tareas del día de hoy.

- Al entrar en foco (con `useFocusEffect`) llama a `loadTasksForDate(today)`.
- Si no hay tareas muestra `<EmptyState>`.
- Si hay tareas muestra `<ProgressBar>` y las tareas agrupadas por prioridad: Alta → Media → Baja → Completadas.
- Cada grupo lleva un `<SectionHeader>` coloreado.
- Botón flotante `<AddTaskFAB>` en la esquina inferior derecha.

**Para cambiar el día mostrado:** modifica el argumento de `loadTasksForDate`.

---

### 6.2 `app/(tabs)/tasks.tsx` — Pestaña "Tareas"

**Qué hace:** Muestra **todas** las tareas con filtros.

- Carga todas las tareas con `loadTasks()` al entrar en foco.
- Dos filas de chips horizontales: filtro de estado (`STATUS_FILTERS`) y filtro de categoría (`CATEGORY_FILTERS`).
- Las tareas se obtienen con `getFilteredTasks()` que aplica ambos filtros.
- Muestra `<EmptyState>` si no hay resultados con el filtro actual.

**Para añadir un filtro nuevo:** añade una entrada a `STATUS_FILTERS` o `CATEGORY_FILTERS` y extiende el tipo `FilterType` o `CategoryFilter` en el store.

---

### 6.3 `app/(tabs)/stats.tsx` — Pestaña "Stats"

**Qué hace:** Muestra estadísticas del historial de tareas.

- Carga **todas** las tareas con `loadTasks()`.
- **Tres tarjetas de stats:**
  - Racha de días (días consecutivos con al menos 1 tarea completada).
  - % de tareas completadas sobre el total.
  - Tareas pendientes.
- **Gráfico de barras semanal:** muestra los últimos 7 días. Cuenta tareas completadas por día usando el campo `completed_at`. La barra de hoy aparece en púrpura sólido, el resto en `accentDim`.
- **Mensaje del día:** array de 7 frases, se selecciona por `getDay()` (día de la semana).
- **Resumen total:** filas con tareas creadas, completadas y pendientes.

**Para añadir más estadísticas:** extiende `getStats()` en el store o calcula directamente desde el array `tasks` dentro del `useMemo` del componente.

---

### 6.4 `app/(tabs)/settings.tsx` — Pestaña "Config"

**Qué hace:** Ajustes y operaciones destructivas.

- Mini-stats en la parte superior (mismo datos que `getStats()`).
- **Toggle de notificaciones:** llama a `cancelAllNotifications()` cuando se desactiva. *Nota: solo cancela las existentes; no reprograma si se reactiva (mejora futura).*
- **Reiniciar el día:** llama a `resetDay()` → desmarca todas y reprograma notifs.
- **Eliminar todo:** llama a `purgeAll()` → cancela notifs y borra la DB.

---

## 7. Modales de tarea

Ambos modales se presentan como sheets deslizables desde abajo (`presentation: 'modal'`, `animation: 'slide_from_bottom'`).

### 7.1 `app/task/new.tsx` — Crear tarea

**Campos del formulario:**

| Campo | Tipo | Default |
|---|---|---|
| Título | TextInput obligatorio | — |
| Descripción | TextInput multilínea, opcional | — |
| **Día** | DayPicker (chips horizontales, 7 días) | Hoy |
| Prioridad | 3 botones: Alta / Media / Baja | Media |

| Campo | Tipo | Default |
|---|---|---|
| Categoría | 4 chips: General / Trabajo / Salud / Personal | General |
| **Notificación** | Toggle on/off | Off |
| Hora notificación | TimePicker (↑↓ horas, ↑↓ minutos en pasos de 5) | 08:00 |
| Repetir recordatorio | Chips: Nunca / 30m / 1h / 2h / 3h | Nunca |

**Al guardar:** llama a `addTask()` del store con todos los campos. Si el título está vacío, muestra un `Alert`.

### 7.2 `app/task/[id].tsx` — Editar tarea

Mismo formulario que el de creación pero:

- Al montar, carga la tarea con `getTaskById(id)` e inicializa todos los estados.
- El DayPicker muestra **14 días** (para permitir mover tareas con más margen).
- Botón de **papelera** en la barra superior que llama a `removeTask()` con confirmación.
- Al guardar llama a `editTask()`. El store se encarga de cancelar/reprogramar la notificación si `reminder_time` o `title` cambiaron.

### 7.3 Componente `DayPicker`

Definido localmente en ambos modales (con leve diferencia: new.tsx muestra 7 días, [id].tsx muestra 14).

```
function buildDays(count: number): { iso: string; label: string; sublabel: string
}[]
// iso:      'YYYY-MM-DD'   → se guarda en la DB
// label:    'Hoy' | 'Mañana' | 'Lun' | 'Mar' ...
// sublabel: '28 abr' (fecha corta)
```

---

## 8. Componentes reutilizables

### `TaskCard`

**Props:** `{ task: Task }`

Muestra una tarjeta de tarea con:

- **Línea de color** en el borde izquierdo según prioridad.
- **Checkbox animado:** al pulsar hace bounce (spring animation). Llama a `toggleTask()`.
- **Contenido:** título con tachado si completada, chips meta (hora, categoría, repetición).
- **Badge de prioridad** a la derecha.
- **Swipe-to-delete:** deslizar a la izquierda >70px revela el botón de borrar con confirmación Alert. Implementado con `PanResponder` + `Animated.Value`.

---

### `AddTaskFAB`

Botón flotante posicionado absolutamente (`bottom: 28, right: 24`). Navega a `/task/new`. Tiene animación spring de escala en press/release.

---

### `ProgressBar`

**Props:** `{ total: number; completed: number }`

Barra horizontal animada con spring animation. Muestra fracción `X / Y`, porcentaje y un mensaje "🎉 ¡Increíble!" cuando el 100% está completado.

---

### `EmptyState`

**Props:** `{ message, subMessage, icon }`

Pantalla vacía centrada con icono Ionicons, título y subtítulo. Usada en las pestañas cuando no hay tareas.

---

### `PriorityBadge`

**Props:** `{ priority: Priority; size?: 'sm' | 'md' }`

Pastilla coloreada con emoji y label. Usa `PriorityConfig` del theme.

---

## 9. Sistema de diseño

**Fichero:** `constants/theme.ts`

Todos los estilos usan **tokens** de este fichero. Nunca se usan valores hardcoded.

### Colores

```
Colors = {
  background:      '#0D0D14',   // fondo de pantalla
  surface:         '#16161F',   // fondo de tarjetas
  surfaceElevated: '#1E1E2E',   // fondo elevado (inputs, chips)
  surfaceActive:   '#252538',   // estado presionado

  accent:          '#7C63FF',   // morado principal (botones, checkboxes activos)
  accentLight:     '#9B85FF',
  accentDark:      '#5A45E0',
  accentDim:       'rgba(124,99,255,0.18)',  // fondo de chips activos
  accentGlow:      'rgba(124,99,255,0.06)',

  success:         '#4ADE80',   // verde (prioridad baja, completadas)
  danger:          '#F87171',   // rojo (prioridad alta, borrar)
  warning:         '#FBBF24',   // amarillo (prioridad media, pendientes)

  text:            '#F0F0FF',   // texto principal
  textMuted:       '#8B8FA8',   // texto secundario / labels
```

```
  textDim:         '#4B4F6A',   // texto terciario / placeholders
}
```

### Espaciado

```
Spacing = { xs: 4, sm: 8, md: 16, lg: 24, xl: 32, xxl: 48 }
```

### Tipografía

```
FontSize = { xs: 11, sm: 13, md: 15, lg: 18, xl: 22, xxl: 28, hero: 34 }
```

### Radio de borde

```
Radius = { sm: 8, md: 12, lg: 16, xl: 24, full: 999 }
```

### PriorityConfig

```
PriorityConfig = {
  high:   { label: 'Alta',  color: Colors.danger,  dimColor: ..., emoji: '🔴' },
  medium: { label: 'Media', color: Colors.warning, dimColor: ..., emoji: '🟡' },
  low:    { label: 'Baja',  color: Colors.success, dimColor: ..., emoji: '🟢' },
}
```

### CategoryConfig

```
CategoryConfig = {
  general:  { label: 'General',  icon: 'list-outline' },
  trabajo:  { label: 'Trabajo',  icon: 'briefcase-outline' },
  salud:    { label: 'Salud',    icon: 'heart-outline' },
  personal: { label: 'Personal', icon: 'person-outline' },
}
```

---

## 10. Flujos principales

### Crear una tarea

```
Usuario pulsa FAB (+)
  → navega a /task/new (modal)
```

```
  → rellena título, elige día, prioridad, categoría
  → opcionalmente activa notificación y elige hora
  → pulsa "Guardar tarea"
    → addTask() [store]
      → DB.createTask() → devuelve id
      → scheduleTaskNotification() → devuelve notification_id
      → DB.updateTask(id, { notification_id })
      → loadTasksForDate(hoy) ó loadTasks()
  → router.back()
```

### Completar una tarea

```
Usuario pulsa checkbox en TaskCard
  → toggleTask(id) [store]
    → DB.toggleTaskComplete(id, true)
    → cancelNotification(notification_id)
    → DB.updateTask(id, { notification_id: null })
    → loadTasksForDate/loadTasks()
```

### Ver estadísticas

```
Usuario abre pestaña Stats
  → useFocusEffect → loadTasks() (carga TODAS las tareas históricas)
  → calcStreak(): recorre días hacia atrás buscando completed_at
  → getLast7Days(): genera array de 7 Date
  → completedOnDay(tasks, day): filtra tasks por completed_at === day
  → getDailyMessage(): elige frase por día de la semana
```

---

## 11. Cómo extender la app

### Añadir una nueva categoría

1. Añade la clave a `CategoryConfig` en `constants/theme.ts`.
2. Añade el tipo a `Category` en `lib/db.ts` y `lib/db.web.ts`.
3. Añade el filtro a `CATEGORY_FILTERS` en `tasks.tsx`.

### Añadir un nuevo campo a las tareas

1. Añade la columna al `CREATE TABLE` y un `ALTER TABLE` de migración en `lib/db.ts`.
2. Replica el campo en `lib/db.web.ts`.
3. Añade el campo a la interfaz `Task` y `NewTask` si aplica.
4. Actualiza `createTask()` y `updateTask()` en ambos ficheros de DB.
5. Añade el UI al formulario en `new.tsx` y `[id].tsx`.

### Cambiar colores del tema

Edita únicamente `constants/theme.ts` → sección `Colors`. El resto de la app lo recoge automáticamente.

### Añadir una nueva pestaña

1. Crea el fichero en `app/(tabs)/nombreTab.tsx`.
2. Añade `<Tabs.Screen name="nombreTab" options={...} />` en `app/(tabs)/_layout.tsx`.

### Añadir un nuevo tipo de notificación

Extiende `lib/notifications.ts`. La función `scheduleTaskNotification` acepta un `Task` completo, por lo que si añades campos a `Task` puedes usarlos para personalizar el mensaje.